

AI4AI-Bench: Benchmarking LLM Agents in Algorithmic Design for Recursive Self-Improvement

Yizhe Chi[†], Wenyi Li, Deyao Hong, Xiaoqiu Wang, Mingju Gao, Kaisen Yang, Bingxiang He, Youjie Zheng, Calvin Xiao, Qinhuai Na[‡]

Navers Lab, Einsia.AI Tsinghua University

[†]Project Lead [‡]Corresponding Author

Abstract. Recursive self-improvement (RSI) asks whether an AI system can improve the process that produces AI systems, so that the next system inherits the improvement. That process is a training algorithm, and it is the part of the work whose gains are inherited: a better objective or update rule changes the exchange rate between compute and capability for every run that follows, including the run that produces the next agent. Whether RSI is feasible therefore turns on whether an agent can design training algorithms. No benchmark isolates that ability: existing suites are won by collecting data or by tuning hyperparameters, and none tells a change to how a run is executed apart from a change to how the model learns. We present AI4AI-Bench, 10 frozen research repositories spanning 10 families of training algorithm, each asking an agent to improve the training algorithm that repository applies to its own model: the agent has 4 hours on one B300 to rewrite the code, the code it leaves behind is then executed afresh for up to 12 hours, and the result is measured by an evaluator that was fixed in advance and is never visible while the code is being written, against the repository’s own algorithm run under the identical procedure. Because the 10 metrics are incommensurable, every task is mapped onto one scale on which 0 is an uninformative model, 0.1 is the algorithm the repository ships, and 1.0 is the task optimum. Across 29 configurations of 6 systems on all 10 tasks the mean score is 0.166, and the best system reaches 0.250: even the strongest closes under a fifth of the distance between the algorithm that was already there and the optimum. The submissions show where that distance went: most never change how the model learns at all, and the minority that do average 0.226 against 0.126 for the rest. More reasoning effort mostly buys the willingness to go there, taking that minority from 8% of submissions to 64% and the mean score from 0.094 to 0.196. We release the task suite, the evaluators and every scored submission, so that the measurement can be repeated as these systems change.

Date: August 21, 2026

Homepage: <https://lab.einsia.ai/ai4ai>

Correspondence: nana@einsia.ai



1 Introduction

Recursive self-improvement (RSI) rests on a loop: a system improves the process that produces its successor, and the successor inherits the improvement. Three levels in that loop can be automated by a coding agent — the *Systems engineering* level (kernels, parallelism, communication), the *Data* level (mixtures, synthesis, filtering), and the *Algorithmic design* level (objectives, update rules, regularization, schedules). Work on the *Systems engineering* level is bounded by the hardware it runs on, since once a kernel reaches the hardware roofline, further gains are no longer possible. Work on the *Data* level is bounded by a finite stock of human text [49], by synthetic corpora that largely re-express what the model already carries [45], and by a power law in which every further halving of the loss gap costs several times more tokens than the halving before it [20, 25]. Work on the *Algorithmic design* level is different in kind: a better objective or update rule changes the exchange rate between compute and capability [19], so every training run that follows — including the run that produces the next agent — inherits the gain. Adam, layer normalization, DPO and GRPO were each paid for once and have been earning since; if RSI is going to compound, most of the compounding has to

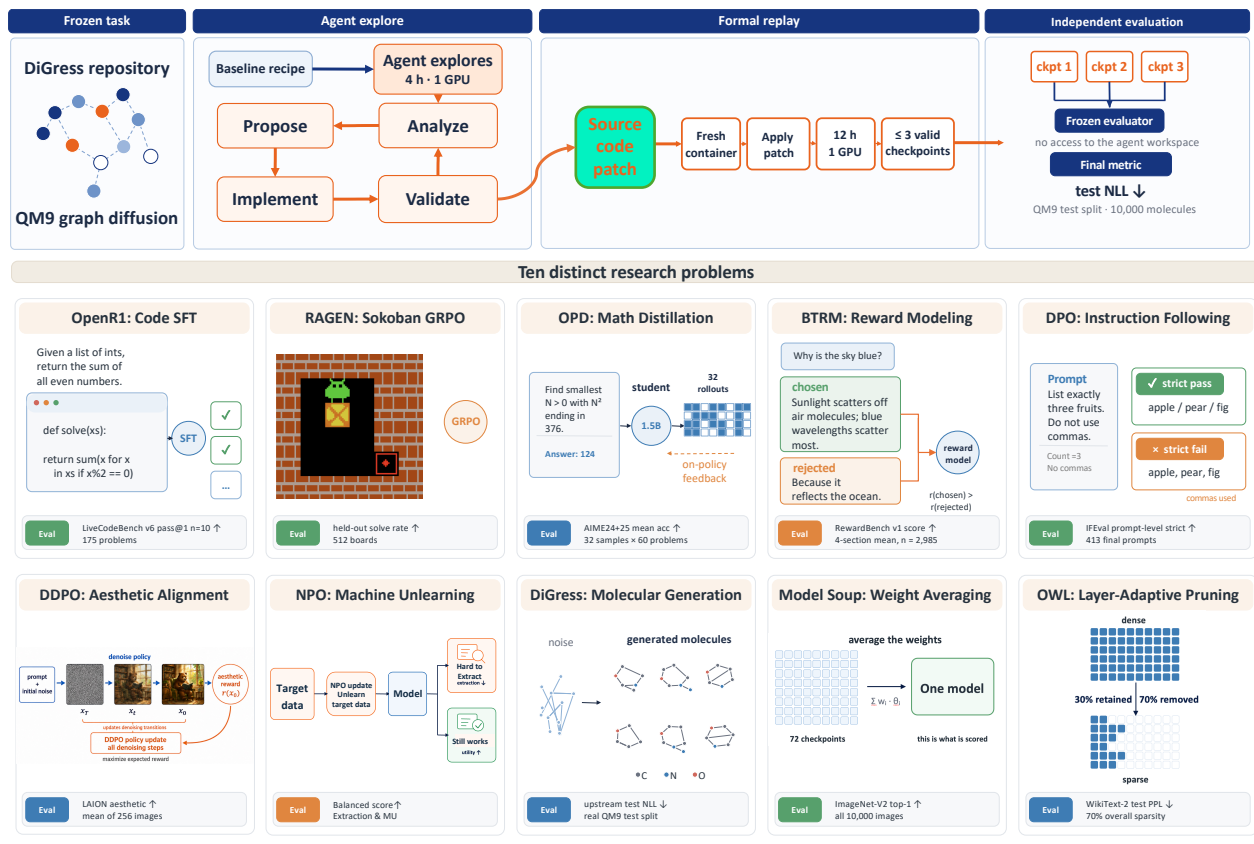


Figure 1 AI4AI-Bench at a glance. *Top:* the lifecycle every task runs through, drawn here with discrete graph diffusion as the running example. The repository, the model it starts from and the cheap proxy metric are frozen; the agent then has four hours on one B300 to read the code, change it, and test each idea against that proxy. What it leaves behind is a source-code patch and nothing else — no weights it trained, no cached state. The patch is applied in a fresh container and run from initialization for up to twelve hours, and at most the three most recent checkpoints are kept. An evaluator fixed before the first run, with no access to the agent’s workspace, scores them, and the cell takes the best of the three under the task’s direction; for the two tasks that do not train, the twelve hours are only a ceiling on the verification stage (§2.3). *Bottom:* the ten frozen repositories, chosen so that between them they cover ten distinct families of training algorithm, each shown with the final metric its own evaluator computes and the asset that metric reads (Table 1).

come from this level.

No benchmark isolates that level. The Kaggle-derived suites ask for a competition submission — predictions over one fixed dataset — so what wins is feature engineering and ensembling while the learning algorithm stays a library call the agent never edits, and nothing in the submission is inherited by the next training run [3, 42]. PostTrainBench states its task end to end, post-training a base model against a released instruction-tuned checkpoint, and its largest levers are which data to assemble and what to initialize from rather than the objective [43]; RSIBench-Data makes that emphasis deliberate, freezing the post-training stack so that only data-centric decisions vary [39]. MLS-Bench comes closest, with 140 tasks in which an agent improves one component of an ML system [36], but the component boundary is handed to it and the score conflates execution-level improvements with changes to the learning algorithm. Closest of all is *autoresearch*, which hands the agent one training file and declares everything in it fair game, architecture and optimizer included [26]; but a five-minute run of a single script is not a research repository, and measured against classical optimizers the agent’s edits behave like hyperparameter search and lose to CMA-ES and TPE [16]. Editing source code is therefore not the same as designing an algorithm, and none of these settings answers the question the loop turns on: did the agent change *how the run was executed*, or *how the model learns*?

The line between the two is not the size of a change but what it touches: a hyperparameter is a number the

training algorithm takes as given, an algorithmic change rewrites the algorithm — the loss it optimizes, the update it applies. The second kind is what a machine learning scientist does at an industrial training system. They read the training dynamics — the loss curve and where it spikes, gradient norms, the entropy of the policy, the divergence from the reference model, the distribution of advantages, the loss broken down by token — and infer from them which part of the algorithm is misbehaving: a policy whose entropy has collapsed, a penalty term that has come to dominate the objective, a reward model saturating on the easy half of its pairs. The diagnosis names a mechanism and the fix changes that mechanism.

In this paper, we propose **AI4AI-Bench**, a benchmark built to isolate the algorithmic design level. It includes 10 research repositories, each representing a distinct family of training algorithms — supervised fine-tuning, multi-turn agentic RL, on-policy distillation, Bradley–Terry reward modeling, preference optimization, diffusion RL, machine unlearning, discrete graph diffusion, weight averaging and one-shot pruning — and asks an agent to improve each repository’s *own* training algorithm, rather than to reach a target somebody else set. Every task carries the same contract. The agent has 4 hours on one B300 GPU to read the repository, change its training code, and test each idea against a fast proxy metric. When the 4 hours are up the agent stops, and the code it leaves behind is trained from a clean start for up to 12 hours. This is the asymmetry a machine learning scientist works under: an idea can be triaged in minutes, but the run that settles it takes a day. The data behind that final measurement is never available during the 4 hours: the agent may consult its proxy as often as it likes, but the evaluation that decides its score is out of reach, exactly as a held-out test set is out of reach of the development loop in any industrial training system.

Results. Across 29 configurations of six systems on all ten tasks the mean score is 0.166 and the best system reaches 0.250, on a scale where 0.1 is the algorithm the repository ships and 1.0 the task optimum. The submissions say where the remaining distance went. Of the 263 that change anything, 141 leave the learning procedure exactly as they found it and move budgets, checkpointing, hyperparameters and capacity instead. The 122 that do reach it — the objective, the supervision signal, the learning rule, the data — average 0.226 against 0.126 for the rest: the algorithmic layer is where the distance gets closed, and most submissions never go there. More reasoning effort moves agents towards it, taking that share from 8% to 64% and the mean score from 0.094 to 0.196, which is most of what effort buys. A machine learning scientist reads the training dynamics, names the mechanism that is failing, and changes that mechanism; few of these submissions do.

In summary, we make the following contributions:

- **A benchmark that isolates the algorithmic level.** Ten research repositories spanning ten families of training algorithms, evaluated under a unified protocol that separates a four-hour agent development window from the up to twelve-hour clean-start training run used for scoring.
- **A measurement of what agents *do*, not only whether they win.** Every submission is classified by what it changes, which is what turns “the agent improved the training algorithm” into a checkable statement rather than a restatement of the score.
- **Revealing a gap between algorithmic exploration and actual improvement.** The algorithmic design level is where the distance to a better algorithm is closed and the one these agents reach least often, and more reasoning effort mostly buys the willingness to reach it.

2 AI4AI-Bench

2.1 Task formulation

Each task presents an agent with a research repository, a base model to start from, and an inexpensive proxy metric that can be evaluated freely during development §2.2. The agent is given 4 hours on one B300 GPU and a single objective: improve the training algorithm implemented in the repository. Its submission is neither a number nor a trained model, but the repository’s source code after the agent’s modifications. After submission, the agent can no longer modify or interact with the code. The submitted repository is then run from scratch under a fixed budget, and the resulting model is scored by a predetermined evaluator (§2.3).

Formally, a task is a tuple (C, a_0, q, m, d) , where C is the repository’s source in its frozen state, a_0 the model it starts from, q an inexpensive proxy available to the agent, m the final metric, and $d \in \{\uparrow, \downarrow\}$ its direction. An

Table 1 The ten AI4AI-Bench tasks. Each freezes a research repository and asks an agent to improve the training algorithm that repository applies to its own model. **Starting model** is the model a_0 the procedure begins with, and **Evaluation metric** the quantity a fixed evaluator computes afterwards, with its direction. Between them the ten cover ten families of training algorithm rather than ten instances of one, and their metrics are incommensurable, which is why no number in this paper averages across rows. [†]Weight averaging and one-shot pruning do no training, and are executed once rather than trained to a horizon.

Task	Algorithm family	Starting model	Evaluation metric
OpenR1	supervised fine-tuning	Qwen2.5-Coder-1.5B-Instruct	LiveCodeBench $\uparrow$
RAGEN	multi-turn agentic RL	Qwen2.5-3B-Instruct	held-out solve rate $\uparrow$
OPD	on-policy distillation	R1-Distill-Qwen-1.5B	AIME 24/25 $\uparrow$
BTRM	Bradley–Terry reward model	Mistral-7B-Instruct-v0.2	RewardBench $\uparrow$
DPO	preference optimization	merged Zephyr/Mistral-7B	IFEval strict $\uparrow$
DDPO	diffusion RL	Stable Diffusion v1.5	aesthetic score $\uparrow$
NPO	machine unlearning	Llama-3.2-1B-Instruct	balance score $\uparrow$
DiGress	discrete graph diffusion	QM9 graph diffusion model	test NLL $\downarrow$
Model Soup [†]	weight averaging	72 CLIP checkpoints	ImageNet-V2 top-1 $\uparrow$
OWL [†]	one-shot pruning	OPT-6.7B dense	WikiText-2 perplexity $\downarrow$

agent observes (C, a_0, q) under an exploration budget $T_e = 4$ hours on one B300 GPU and returns a rewritten source C' ; it never evaluates m . Execution of C' under a verification budget $T_v = 12$ hours yields a model $a(C')$, and the score of the resulting cell is

$$s(C') = m(a(C')),$$

where m is computed by an evaluator E that is fixed in advance and has no access to the agent’s execution environment. Applying the identical procedure to the unmodified source gives the baseline $s(C) = m(a(C))$, and a submission constitutes an improvement precisely when $s(C') \succ_d s(C)$, that is, $s(C') > s(C)$ when $d = \uparrow$ and $s(C') < s(C)$ when $d = \downarrow$. The two executions differ in C' against C and in nothing else: the hardware, the budget, the evaluator and the evaluation asset are common to both.

What is held fixed is the measurement — the evaluation asset, the final metric, and the evaluator that computes it — and what is open is the method. The agent may rewrite the training loop, the objective, the optimizer, the data pipeline, the schedule, or all of them; the single line it may not cross is the evaluation itself. Fixing only the outcome and its measurement asks the question this paper is about: whether an agent can find a way to improve the system at all, and which part of the system it reaches for when nothing constrains it.

2.2 AI4AI-Bench algorithmic tasks

The suite is ten frozen research repositories, listed in Table 1 and chosen so that between them they cover ten families of training algorithm rather than ten instances of one: supervised fine-tuning, multi-turn agentic RL, on-policy distillation, Bradley–Terry reward modeling, preference optimization, diffusion RL, machine unlearning, discrete graph diffusion, weight averaging, and one-shot pruning. Each was admitted on three properties the design depends on. It must ship a training algorithm its authors actually run, not a tutorial or a toy; it must ship a frozen starting model, so that there is something the agent is improving *from*; and its metric must be recomputable, reproducibly, under a half-day budget on a single B300, or the twelve-hour measurement in §2.3 could not be run at all — let alone once per cell.

Two of the ten do no training. Weight averaging combines a bank of checkpoints that are handed over as data, and one-shot pruning removes weights from a released model in a single pass; neither has a training horizon that could be extended or shortened. They are kept because the algorithmic question is just as real in them — which checkpoints to combine and how, which weights to remove and by what criterion — and because a suite that quietly dropped them would be a suite about training loops rather than about algorithm design. The protocol treats them differently, and so does the baseline.

The ten metrics are incommensurable: an aesthetic score, a perplexity, a solve rate, a pass rate, an unlearning balance score. We therefore never average them. Every aggregate in this paper is a rate over per-task

comparisons, and every per-task number is reported in its own units against its own baseline.

2.3 Protocol

Exploration is the same everywhere. For four hours the agent works inside the repository on one B300, free to read it, edit it, launch training runs of its own, and consult the fast proxy metric without limit. When the four hours are up it stops, and the code it leaves behind is the submission. Nothing else crosses forward: no weights it trained, no cached state, no notes to itself — only the source.

What happens next depends on the task. For the eight tasks that train, the submitted source is executed from initialization until it terminates or twelve hours elapse, whichever comes first; the three most recent checkpoints are then scored, and the cell takes the best of them under the task’s direction. For the two that do not train, the submitted code is simply executed once to produce its model — an averaged model, a pruned model — which is scored directly; there the twelve hours are only a ceiling on the verification stage, not a training budget being spent.

The boundary between what the agent may measure and what decides its score is the load-bearing property of the whole design. During its four hours the agent may query the fast proxy as often as it likes; the final metric is computed afterwards, from source it can no longer touch, by an evaluator frozen before the first run. The separation is one of *access and timing* rather than of sample disjointness: on some tasks the cheap proxy is drawn from the same corpus the final evaluation uses, because running the full evaluation as a proxy would cost more than the exploration budget allows. So what the boundary guarantees is that no agent could score a candidate under the metric that decides its result — not that it never saw a row that metric would later read.

2.4 Baselines

Calling a change an improvement requires something to compare it against, and the comparison this paper reports is deliberately the strictest one available: the repository’s own algorithm, given exactly what the agent was given. For a task that trains, the baseline is the repository’s committed code executed under the identical procedure and the same twelve-hour budget, and measured by the same fixed evaluator on the same asset. For the two tasks that do not train, it is the repository’s recipe executed as it stands, scored the same way. In both cases the only difference between the baseline and a submission is the source code itself — same hardware, same budget, same evaluator, same asset — which is what makes a win attributable to the change the agent made rather than to the resources it was given.

This is a harder bar than it may look, and a different one from what neighbouring benchmarks use. It is not a published number, so it cannot have been tuned on a different evaluation than ours; it is not an official instruction-tuned release, so beating it is not a matter of assembling more data than the authors had; and it is not a human expert attempt, so it makes no claim about where human performance lies. It is the answer to one question only: does the agent’s code produce a better model than the code that was already there, run under identical conditions?

2.5 Scoring

Whether a submission beat the baseline is one bit, and one bit is too little for either use this suite is meant to serve. Across tasks it makes a strong submission and a marginal one indistinguishable, and it hides the difference between failing narrowly and failing completely. More importantly, a benchmark of this shape is a natural environment for training agents by reinforcement learning, and a binary outcome is a sparse reward: it gives no gradient between the many submissions that do not beat the baseline, which on most tasks is where most submissions are. What is needed is a dense score — one that separates submissions everywhere along the range a metric can occupy, not only at the point where it crosses a reference.

Each task is therefore equipped with a *progress coordinate* φ , a strictly increasing function of quality that absorbs the metric’s direction, together with three reference points in the metric’s own units: the uninformative model $x_{\perp}$, the baseline $x_b = s(C)$, and the optimum x^* . Writing $\phi_{\perp}, \phi_b, \phi^*$ for their images under φ , every

task is scored by the same function,

$$\sigma(x) = \begin{cases} 0.1 \frac{\varphi(x) - \phi_{\perp}}{\phi_b - \phi_{\perp}}, & \varphi(x) \leq \phi_b, \\ 0.1 + 0.9 \frac{\varphi(x) - \phi_b}{\phi^* - \phi_b}, & \varphi(x) > \phi_b, \end{cases} \quad (1)$$

clipped to $[0, 1]$, with $\sigma = 0$ for a submission that returned no model at all. The two branches meet at $\sigma = 0.1$: matching the recipe the repository ships is the pivot of the scale, what lies below it measures how far a submission fell short of that, and what lies above it measures how much of the remaining distance to the optimum it closed.

Only the triple $(\varphi, x_{\perp}, x^*)$ changes from task to task, and each element is fixed by the metric rather than by the results. The optimum x^* is the metric’s best attainable value: a rate of 1, a preference score of 100, a perplexity of 1, a negative log-likelihood of 0. The uninformative point $x_{\perp}$ is what a model carrying no information about the task would score — 0 for a rate, the chance level of 50 for pairwise preference, the uniform predictor for a likelihood — which is why it is not always 0 in the metric’s own units. And φ is the identity wherever the metric is already a linear utility, which includes the rates, the aesthetic head and the negative log-likelihood, and is $-\log$ for a perplexity, since a perplexity is the exponential of a cross-entropy and only its logarithm lies on the same scale as the likelihood tasks. Without that one transformation, a submission taking OWL from 53.4 to 16.2 would read as having closed 71% of the distance to the optimum, where the correct figure is 30%: a perplexity of 53.4 is a cross-entropy of $\log 53.4 = 3.98$ nats above a perfect predictor and one of 16.2 is 2.79 nats above it, so the submission removed 1.19 of the 3.98 nats that separated the repository’s own recipe from the optimum.

3 Experiments

3.1 Setup

A model cannot be separated from the framework that runs it, so what is under test here is not a model but the whole combination of model, harness and reasoning effort, which we call a *system*. We evaluate six systems: three GPT-5.6 variants — Sol, Terra and Luna — under Codex at all six effort levels; two Claude 5 variants, Opus 5 and Sonnet 5, under Claude Code at the five levels that harness exposes; and Kimi K3 under Claude Code at its highest. That is 29 configurations, each attempting all ten tasks, for 290 cells.

3.2 Results

The whole study sits in the lowest fifth of the scale. The mean score over the 290 cells is 0.166, the strongest system averages 0.250 (Figure 2), and the single best configuration in the study, Claude Opus 5 at medium effort, averages 0.288. Against a scale on which 0.1 is the algorithm each repository already ships and 1.0 is the task optimum, that is a fifth of the distance at the very top and well under a tenth on average. In the other direction, 124 of the 290 cells fall below 0.1: more than two fifths of the attempts leave the repository with something worse than what it had. None of this is legible in the raw units of Table 2, where lifting weight averaging by 0.015 of top-1 accuracy and taking a perplexity from 53.4 to 13.0 are both simply cells that beat a baseline.

Systems are ordered, and the ordering is compressed. Figure 2 separates the six cleanly — Claude Opus 5 at 0.250, then GPT-5.6 Sol at 0.191, Kimi K3 at 0.174, Claude Sonnet 5 at 0.145, GPT-5.6 Terra at 0.135 and GPT-5.6 Luna at 0.117. But the entire range lies inside the bottom quarter of the scale, so the choice of system moves the number without moving the regime: the best system’s average is closer to the weakest system’s than it is to the optimum it was asked to approach.

Spend does not explain the result. The cost column of Table 2 ranges about ninefold across systems on the same harness — a median configuration costs \$434 for Sol and \$48 for Luna — and the ordering it induces is not the ordering of the scores. Opus 5 leads the study at a median of \$181, under half of what the second-placed system spent, and Sonnet 5 spends about twice Luna’s budget for a 0.028 difference. Whatever is separating these systems, it is not how much exploration they bought.

Table 2 Every configuration on every task, in the metric’s own units. Rows are the 29 (model, harness, effort) configurations; \$ is what the four hours of exploration cost. *Baseline* is the score of the repository’s own code under the identical procedure (§2.4). Backgrounds report the mapped score of §2.5: below 0.1 is worse than that baseline, 0.1 to 0.4 beats it, above 0.4 closes more than a third of the distance to the optimum. **Bold** marks the best cell in a column, and a dash a configuration that returned nothing trainable. The last three rows give the ladder each column is scored on.

System	Harness	Effort	OpenR1↑	RAGEN↑	OPD↑	BTRM↑	DPO↑	DDPO↑	NPO↑	DiGress↓	Soup↑	OWL↓	Cost (USD)
<i>Baseline</i>	–	–	0.127	0.170	0.436	74.9	0.424	5.84	0.887	65.8	0.686	53.4	–
* Claude Opus 5	Claude Code	low	0.138	1.00	0.421	71.6	0.467	12.1	0.997	65.4	0.696	13.0	181
* Claude Opus 5	Claude Code	medium	0.121	1.00	0.432	77.1	0.622	8.98	1.01	65.8	0.696	13.4	166
* Claude Opus 5	Claude Code	high	0.127	1.00	0.449	71.9	0.615	8.42	1.01	65.7	0.694	13.2	181
* Claude Opus 5	Claude Code	xhigh	0.128	0.211	0.440	64.4	0.421	14.4	1.02	65.8	0.696	13.3	185
* Claude Opus 5	Claude Code	max	0.125	0.234	0.392	75.7	0.477	17.7	1.03	64.8	0.694	13.0	195
* Claude Sonnet 5	Claude Code	low	0.108	0.043	0.432	69.3	0.404	6.13	0.969	66.3	0.698	54.0	93
* Claude Sonnet 5	Claude Code	medium	–	0.000	0.427	71.5	0.400	5.86	0.969	65.4	0.693	19.7	98
* Claude Sonnet 5	Claude Code	high	0.113	0.895	0.427	74.5	0.540	5.82	0.733	66.5	0.694	21.7	96
* Claude Sonnet 5	Claude Code	xhigh	0.114	0.240	0.421	38.3	0.506	4.49	0.963	66.6	0.694	21.2	101
* Claude Sonnet 5	Claude Code	max	0.113	0.232	0.428	74.9	0.542	6.15	0.948	66.2	0.699	14.3	109
κ Kimi K3	Claude Code	max	0.099	0.186	0.405	74.1	0.542	9.04	0.986	65.2	0.694	14.3	30
⊗ GPT-5.6 Sol	Codex	none	0.115	0.092	0.426	75.2	0.400	5.82	0.994	67.0	0.694	20.9	339
⊗ GPT-5.6 Sol	Codex	low	0.127	0.883	0.437	74.0	0.431	–	0.746	66.9	0.694	14.9	337
⊗ GPT-5.6 Sol	Codex	medium	0.124	–	0.430	73.7	0.494	6.83	0.998	65.5	0.695	15.8	449
⊗ GPT-5.6 Sol	Codex	high	0.126	1.00	0.424	74.0	0.458	5.77	0.966	66.0	0.697	13.3	420
⊗ GPT-5.6 Sol	Codex	xhigh	0.125	0.549	0.418	73.8	0.482	5.92	1.03	63.7	0.692	13.9	521
⊗ GPT-5.6 Sol	Codex	max	0.126	1.00	0.429	73.9	0.436	8.98	0.998	65.9	0.701	17.4	626
⊗ GPT-5.6 Terra	Codex	none	0.096	0.221	–	75.2	0.443	5.54	0.713	–	–	–	4
⊗ GPT-5.6 Terra	Codex	low	0.127	–	–	72.9	0.421	5.54	0.952	65.3	0.694	20.4	35
⊗ GPT-5.6 Terra	Codex	medium	0.126	0.184	0.443	74.1	0.448	5.85	0.921	64.3	0.694	20.2	43
⊗ GPT-5.6 Terra	Codex	high	0.108	0.086	–	75.4	0.494	6.62	0.997	–	0.694	15.2	214
⊗ GPT-5.6 Terra	Codex	xhigh	0.127	0.266	0.433	70.9	0.438	5.96	0.923	65.6	0.694	16.2	229
⊗ GPT-5.6 Terra	Codex	max	0.127	0.998	0.435	66.9	0.460	6.01	0.954	69.9	0.694	15.2	346
⊗ GPT-5.6 Luna	Codex	none	0.126	–	0.432	75.3	0.407	5.11	0.936	–	0.694	42.9	17
⊗ GPT-5.6 Luna	Codex	low	0.115	0.154	–	74.5	0.472	5.54	–	–	0.694	51.0	7
⊗ GPT-5.6 Luna	Codex	medium	0.125	–	0.438	75.8	0.492	5.56	0.748	66.2	0.694	22.4	30
⊗ GPT-5.6 Luna	Codex	high	0.122	0.125	0.424	74.0	0.402	5.62	0.990	67.4	0.694	29.2	66
⊗ GPT-5.6 Luna	Codex	xhigh	0.126	–	0.427	75.4	0.564	5.74	0.957	68.3	0.692	16.2	110
⊗ GPT-5.6 Luna	Codex	max	0.116	–	0.433	74.4	0.533	6.53	0.945	65.3	0.694	19.7	108
Optimum x^*			1	1	1	100	1	23.23	2.08	0	1	1	–
Uninformative $x_{\perp}$			0	0	0	50	0	–12.76	0	∞	0	∞	–
Coordinate φ			x	x	x	x	x	x	x	$-x$	x	$-\log x$	–

4 Analysis

4.1 Most submissions change how the run goes, not how the model learns

Everything so far has been a score, and a score says only whether a submission worked. It does not say whether the agent designed an algorithm or tuned one that was already there, and those are the two outcomes this paper exists to separate. Counting lines does not separate them either: a one-line diff can replace a learning rule, and a thousand-line refactor can leave the training procedure exactly as it was. The distinction has to be read off the submitted code, by asking which part of the training procedure it modifies.

We group every change into eight families on two sides of that line (Table 4). The grouping is assigned by a separate language model reading each submitted diff against the definitions that follow. Four change *how this run goes*: how long it trains and how often it saves; the training hyperparameters, such as the learning rate or the batch size; which of the checkpoints it produced to keep; and how much trainable capacity to attach and where, such as the rank and placement of an adapter. Four change *how the model learns*: the loss it optimizes, by adding, removing or reweighting a term; the supervision it learns from, by introducing a signal the procedure did not have before; the update rule itself, replaced by a different one; and the data the procedure trains on. The families are not exclusive — a submission matches 3.13 of them on average, since changing a loss usually drags a hyperparameter along with it — so we report how many submissions reach a side rather than assigning each to a single family.

Of the 280 submissions, 17 made no change that could be classified. Of the remaining 263, 141 stay entirely on the run side and only 122 touch how the model learns. Four hours, a whole repository, and a task statement that says in as many words to improve this training algorithm, and more than half of the submissions never reach that layer.

Do the submissions that reached it do better? On the scale of §2.5 they do, and by a wide margin: submissions that touch the learning procedure average 0.226 against 0.126 for those that stay on the run side, a gap of

Table 3 The same cells after the ladder of §2.5. 0.1 is the repository’s own recipe and 1.0 the task optimum, so a score states how much of the remaining distance a submission closed; a configuration that returned nothing scores 0. Backgrounds and bold follow Table 2; per-system and per-task means are in Figure 2.

System	Harness	Effort	OpenR1	RAGEN	OPD	BTRM	DPO	DDPO	NPO	DiGress	Soup	OWL	avg
* Claude Opus 5	Claude Code	low	0.111	1.000	0.097	0.087	0.167	0.424	0.182	0.105	0.129	0.420	0.272
* Claude Opus 5	Claude Code	medium	0.095	1.000	0.099	0.179	0.409	0.263	0.195	0.100	0.129	0.413	0.288
* Claude Opus 5	Claude Code	high	0.100	1.000	0.121	0.088	0.398	0.234	0.195	0.101	0.123	0.416	0.278
* Claude Opus 5	Claude Code	xhigh	0.101	0.144	0.106	0.058	0.099	0.543	0.203	0.100	0.129	0.414	0.190
* Claude Opus 5	Claude Code	max	0.098	0.169	0.090	0.129	0.183	0.714	0.207	0.114	0.123	0.420	0.225
* Claude Sonnet 5	Claude Code	low	0.085	0.025	0.099	0.077	0.095	0.115	0.161	0.099	0.134	0.099	0.099
* Claude Sonnet 5	Claude Code	medium	0.000	0.000	0.098	0.086	0.094	0.101	0.162	0.105	0.120	0.326	0.109
* Claude Sonnet 5	Claude Code	high	0.088	0.886	0.098	0.098	0.281	0.100	0.083	0.099	0.123	0.304	0.216
* Claude Sonnet 5	Claude Code	xhigh	0.089	0.176	0.097	0.000	0.228	0.093	0.157	0.099	0.123	0.309	0.137
* Claude Sonnet 5	Claude Code	max	0.088	0.167	0.098	0.100	0.284	0.116	0.146	0.099	0.137	0.398	0.163
κ Kimi K3	Claude Code	max	0.078	0.117	0.093	0.097	0.284	0.266	0.175	0.108	0.123	0.398	0.174
⊙ GPT-5.6 Sol	Codex	none	0.091	0.054	0.098	0.111	0.094	0.100	0.180	0.098	0.123	0.312	0.126
⊙ GPT-5.6 Sol	Codex	low	0.100	0.873	0.102	0.096	0.111	0.000	0.084	0.098	0.123	0.389	0.198
⊙ GPT-5.6 Sol	Codex	medium	0.097	0.000	0.099	0.095	0.209	0.151	0.183	0.104	0.126	0.376	0.144
⊙ GPT-5.6 Sol	Codex	high	0.099	1.000	0.097	0.096	0.153	0.100	0.159	0.100	0.132	0.414	0.235
⊙ GPT-5.6 Sol	Codex	xhigh	0.098	0.511	0.096	0.096	0.191	0.104	0.207	0.129	0.117	0.405	0.195
⊙ GPT-5.6 Sol	Codex	max	0.099	1.000	0.098	0.096	0.119	0.263	0.183	0.100	0.143	0.354	0.245
⊙ GPT-5.6 Terra	Codex	none	0.075	0.155	0.000	0.111	0.130	0.098	0.080	0.000	0.000	0.000	0.065
⊙ GPT-5.6 Terra	Codex	low	0.100	0.000	0.000	0.092	0.099	0.098	0.149	0.107	0.123	0.318	0.109
⊙ GPT-5.6 Terra	Codex	medium	0.099	0.115	0.111	0.097	0.138	0.101	0.125	0.120	0.123	0.320	0.135
⊙ GPT-5.6 Terra	Codex	high	0.085	0.051	0.000	0.118	0.209	0.140	0.183	0.000	0.123	0.384	0.129
⊙ GPT-5.6 Terra	Codex	xhigh	0.100	0.204	0.099	0.084	0.122	0.106	0.127	0.103	0.123	0.370	0.144
⊙ GPT-5.6 Terra	Codex	max	0.100	0.998	0.100	0.068	0.156	0.109	0.150	0.094	0.123	0.384	0.228
⊙ GPT-5.6 Luna	Codex	none	0.099	0.000	0.099	0.114	0.096	0.096	0.137	0.000	0.123	0.149	0.091
⊙ GPT-5.6 Luna	Codex	low	0.091	0.091	0.000	0.098	0.175	0.098	0.000	0.000	0.123	0.110	0.079
⊙ GPT-5.6 Luna	Codex	medium	0.098	0.000	0.103	0.132	0.206	0.099	0.084	0.099	0.123	0.297	0.124
⊙ GPT-5.6 Luna	Codex	high	0.096	0.073	0.097	0.096	0.095	0.099	0.177	0.098	0.123	0.237	0.119
⊙ GPT-5.6 Luna	Codex	xhigh	0.099	0.000	0.098	0.118	0.319	0.100	0.152	0.096	0.117	0.370	0.147
⊙ GPT-5.6 Luna	Codex	max	0.091	0.000	0.099	0.098	0.270	0.136	0.143	0.107	0.123	0.326	0.139

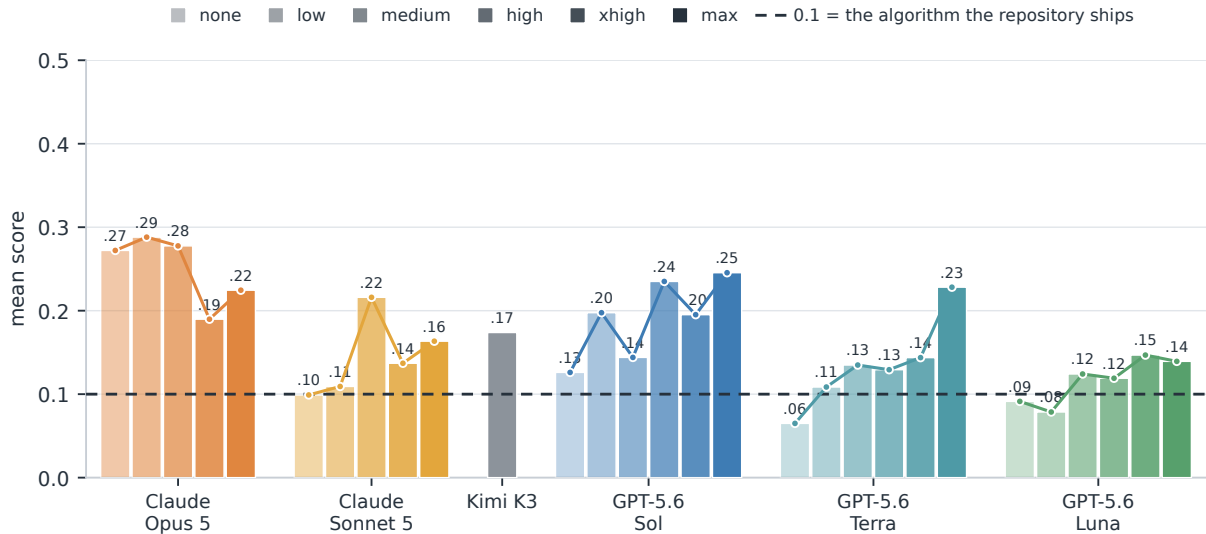


Figure 2 Mean score by system and reasoning effort. One group per model, one bar per effort level, each the mean of that configuration’s ten task scores; deeper colour is more effort. The dashed line at 0.1 is the algorithm each repository already ships. Kimi K3 was run at a single level. Every system in the study sits inside the lowest fifth of the scale, and no system rises monotonically with effort.

0.100 against a standard error of 0.022. It is not the artifact of a single task — dropping agentic RL, where imitation learning lifts the whole column, still leaves 0.182 against 0.128 — nor of a single system, since the ordering holds within four of the five models in this corpus. It is also not a randomized comparison: the systems that reach the learning procedure more often are the stronger ones to begin with, so the gap is the difference between the submissions that go there and the submissions that do not, rather than the effect of going there.

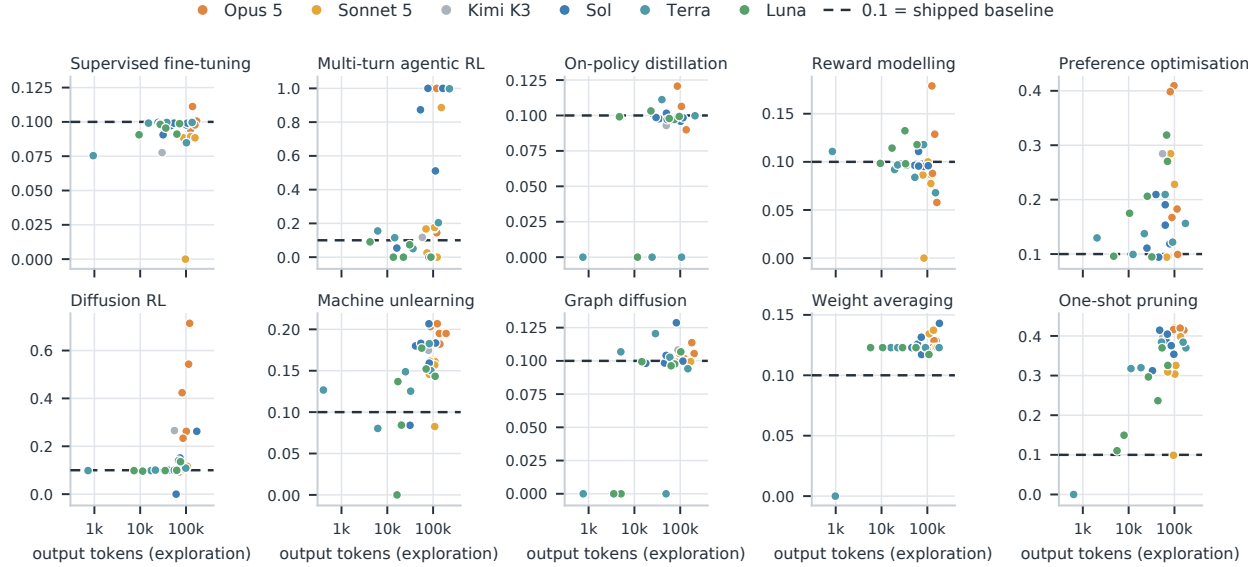


Figure 3 Mapped score against exploration spend, one panel per task. Each point is one of the 29 configurations; x is the number of output tokens it generated during its four-hour exploration of that task (log scale), y is its score after the mapping of Table 3, and the dashed line is 0.1, the score of the repository’s own shipped algorithm. We read spend in output tokens rather than total tokens because input counts are inflated by each harness’s context replay and are not comparable across systems, while output tokens are what the reasoning-effort setting actually moves. Each panel keeps its own y range, so that the columns sitting on the 0.1 line stay readable; that line is always within range. Missing points are configurations that returned no scorable model.

Table 4 What the submissions change. The 263 submissions that changed anything that could be classified, out of 280 (§3.1); Kimi K3 is outside this corpus. Families are not exclusive — a submission matches 3.13 of them on average — so the shares do not sum to one, and the line that matters is how many submissions reach the learning side at all. The lower panel gives that share by reasoning effort.

Family		n	share
run	how long it trains, how often it saves	253	96.2%
	the training hyperparameters	195	74.1%
	which checkpoint to keep	105	39.9%
	how much trainable capacity, and where	73	27.8%
learning	the loss it optimizes	87	33.1%
	the supervision it learns from	66	25.1%
	the update rule itself	23	8.7%
	the data it trains on	21	8.0%
any learning family		122	46.4%
<i>run side only</i>		141	53.6%

<i>reaching the learning side</i>	none	low	medium	high	xhigh	max
share of submissions	8%	39%	33%	49%	65%	64%

Read together, the two numbers say that the algorithmic layer is where the distance actually gets closed, and that most submissions never go to it. What separates a submission that reaches that layer from one that does not is not effort spent but a step taken first: reading the training dynamics as a specific failure mechanism, and then addressing that mechanism.

4.2 Reasoning effort buys nerve, and nerve is what pays

Is there anything that pushes agents down to that layer? The setup has exactly one knob that can be turned on its own, the reasoning-effort level, and taking it from the lowest setting to the highest shows clearly what it buys and what it does not.

It buys nerve. The share of submissions that touch the learning algorithm rises from 8.0% to 64.0%. At low effort the submissions move budgets, logging and optimization knobs; at high effort they operate on objectives, replace learning rules, and add supervision to the procedure.

It buys attempts. Within the Codex grid, the only one that exposes the lowest setting, the median configuration goes from 4 evaluations inside its four hours to 16, from 18 edited lines to 246, and from 11k output tokens to 109k. Cost follows: the median exploration cost per task rises from \$1.69 to \$34.60. The exploration stage of the whole evaluation consumed \$5,334 of API calls, with Kimi K3 converted from Chinese yuan and with neither the GPU hours of the twelve-hour runs nor the evaluator’s compute included.

It buys completion. Of the 19 cells that score zero, 8 ended their four hours without a usable patch — four of them workspaces the host found empty after the agent exited early — and 11 submitted a complete patch that started its twelve-hour run but finished with nothing satisfying the contract, most often no loadable merged model written to disk. All 19 terminated normally: the failure is in what the agent submitted. They concentrate at low effort, with 12 of the 19 at the two lowest levels and one each at the two highest.

And it does buy a result, though a small one in absolute terms. On the scale of §2.5 the mean rises from 0.094 at the lowest level to 0.196 at the highest, a gap of 0.102 against a standard error of 0.027; within the Codex grid, where the harness is held fixed, it rises at every step, 0.094 to 0.204. Set beside the eightfold rise in submissions that reach the learning procedure, the score roughly doubles, and 0.196 is still only a tenth of the way from the shipped algorithm to the optimum. Reasoning effort works, then, by making an agent attempt the thing that pays rather than by making the attempt itself better: it brings more agents to the loop that matters — reading the training dynamics, naming the mechanism that is failing, changing it — and what they gain is about what arriving there is worth.

4.3 What the submissions that reached the algorithmic layer did

Among the 122 submissions that touched how the model learns, a few changed not one step of the procedure but what the task was taken to be. Three are worth reading in full, and they come from three different tasks.

Turning a task that does no training into one that does. One-shot pruning is defined to do exactly one thing: score which weights to remove, remove them once, stop. The repository’s own procedure leaves a perplexity of 53.4. One submission replaced it with a three-stage pipeline — a different rule for selecting and updating the surviving weights, then a round of layerwise distillation, then a masked knowledge-distillation fine-tune of the whole model (AdamW, 666 steps, cosine decay) — and brought the perplexity to just over 13. Its notes record a diagnosis along the way: a first attempt scored 572, absurdly bad, because the weight-allocation step propagated activations forward through the network and overwrote layer 0’s input in place, so the pruning step was reading layer 31’s activations.

Turning a closed form into an optimization problem. Weight averaging ships a uniform mean over 72 candidate models. One submission first built itself an instrument: the relevant tensors of all 72 models packed into a single matrix resident in GPU memory and the proxy images preprocessed and held, so that loading a coefficient vector and scoring it became one matrix multiplication and one forward pass — 0.38 seconds, against roughly 190 before. On that instrument it ranked five methods, all measured on its own rig: best single model 0.6935, uniform average 0.6880, top- k by accuracy 0.6945, greedy soup 0.7025, and coefficients learned directly by cross-entropy with Adam 0.7020, the last two tied. It also recorded two routes that did not work: extrapolating along a single direction collapses accuracy, and a logit-ensemble proxy does not rank candidates reliably.

Replacing reinforcement learning with imitation learning. Multi-turn agentic RL ships GRPO. The submissions that reached a perfect score judged that on this task it pays to learn from the optimal solution first: generate boards in quantity, label every step with its optimal move, and fine-tune on that supervision; one went further with DAGGER, letting the policy walk and adding the correct answer wherever it went.

The three have one thing in common. Each built something measurable before acting: a solver to establish the task’s ceiling, an evaluation rig five hundred times faster than the one it was given, a localisation of which layer’s activations were being overwritten. This is exactly the capability §4.1 finds missing — and among 263 submissions it is the exception.

5 Related work

System-level AI improvement. Autonomous ML-engineering benchmarks ask agents to enter a competition, operate a research repository, or iterate on an executable ML system [3, 42, 47], and a growing body of work studies the search and exploration strategies these agents use [48]. Such settings exercise data loading, feature construction, model selection, hyperparameter tuning, debugging, and ensembling, but they reward a fixed competition output or a requested system change rather than isolating a change to the learning rule. Kernel generation makes the level explicit: agents write, profile, verify, and optimize GPU programs, with recent settings extending from isolated PyTorch operators [9, 41] to hardware feedback in the loop [61], robust verification and benchmarking [29], production traces [58], dedicated agent harnesses [2], and agents trained specifically for kernel generation [12]. Self-evolving engineering environments optimize deployable designs under automated verifiers in the same spirit [10]. All of this improves how an AI computation is executed; the artifact may be reusable, but the objective and the update that produce the model remain fixed.

Data-centric AI improvement. Data-centric methods make the corpus the object of optimization [59]: domain mixtures are reweighted [14, 54], instruction data is filtered, selected, or repaired [6, 7, 32, 53], and instruction or preference supervision is synthesized outright [11, 51, 55]. Several of these use gradients or optimization internally, yet their output is a selected, corrected, reweighted, or generated dataset that an otherwise unchanged trainer consumes; what the successor inherits is data, not a learning rule. Recent systems move these decisions inside an agent loop: a student is retrained and its weaknesses steer the next round of generation [27], an agent assembles a specialization curriculum [35], or the post-training stack is frozen outright so that only data-centric decisions vary, as in RSIBench-Data [39]. PostTrainBench opens post-training end to end, yet data assembly and initialization remain among its largest levers [43]. This line asks whether an agent can improve *what* a successor learns from; we ask whether it can improve *how* that successor learns.

Algorithmic AI improvement. Automated experimental search long predates LLM agents: random search, automated pipeline selection, multi-fidelity optimization, and population-based training efficiently choose algorithms, configurations, or schedules inside researcher-specified spaces [1, 13, 17, 22, 30]. They are essential baselines for automated research, but changing a learning rate, a batch size, or a schedule is not by itself algorithm design. AlphaTensor, AlphaDev, and FunSearch do produce new algorithms and programmatic heuristics [15, 38, 44]; they do so in compact, engineered search spaces with mechanically checkable evaluators, rather than by diagnosing a training procedure embedded in a research repository. LLM research systems broaden the loop to idea generation, code search, experimentation, reflection, and paper writing [5, 23, 24, 33, 56], and related benchmarks score paper replication, execution of experiments extracted from papers, or open-ended research produced from a workshop call [4, 28, 46]; these works cover increasingly complete research processes, but their endpoint is a competition solution, a replicated experiment, or a paper-quality artifact, and that endpoint alone does not establish that the agent produced a better training algorithm. The closest benchmarks instead begin with an existing AI system and ask an agent to improve it [8, 21, 36, 40, 52, 60]. They permit algorithmic work, and they repeatedly find tuning and engineering easier than method invention. Their scores, however, aggregate gains from execution, data, capacity, hyperparameters, and learning rules, or else hand the component boundary to the agent in advance, as MLS-Bench does; they therefore measure whether a system improved, not which level produced the gain. Source-level self-improvement exposes the same ambiguity. `autoresearch` opens architecture, optimizer, and training-loop code [26], yet a controlled comparison finds its edits behave largely as hyperparameter optimization [16]; elsewhere, models are trained specifically for self-improvement in ML engineering [57], or asked to construct agents of their own [34]. Opening source code is thus not the same as isolating an algorithmic intervention. A second difficulty is that what an agent optimizes need not be what a benchmark intends: agents exploit graders and reward channels [18, 37, 50], which is why automated R&D settings increasingly monitor what an agent did alongside what it scored [31]. We therefore take repository source as

the submission, remove the agent before evaluation, rerun that source from a clean start, and classify what the patch changed. The score establishes whether the repository improved; the patch establishes whether the agent changed execution, data, or the training algorithm itself.

6 Conclusion

Recursive self-improvement compounds through the algorithmic link, and AI4AI-Bench is built to measure that link on its own: ten frozen research repositories, each asking an agent to improve the training algorithm it already applies to its own model, with four hours to write code, twelve to run what was written, and an evaluation the agent never sees. Across 29 configurations of six systems on all ten tasks the mean score is 0.166 on a scale where the algorithm the repository already ships is 0.1 and the task optimum is 1.0, and the best system reaches 0.250. The submissions say where the rest of that distance went: of the 263 that changed anything, 141 never touch how the model learns at all, and the 122 that do average 0.226 against 0.126 for the rest — the algorithmic layer is where the distance is closed, and most submissions never go to it. More reasoning effort mostly buys the willingness to go, taking that minority from 8% of submissions to 64% and the mean score from 0.094 to 0.196, which still leaves the strongest setting a tenth of the way past the algorithm it started from. What today’s agents do at the algorithmic link, then, is recover a competent default rather than design past one; whether that changes is the measurement this benchmark exists to keep taking.

References

- [1] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(10):281–305, 2012.
- [2] Joshua Brodsky, Dhruvid Kumar, Savini Kashmira, Jayanaka Danatanarayana, Jason Mars, Krisztian Flautner, and Lingjia Tang. Kernel forge: An agent harness for LLM-based generation and optimization of CUDA kernels. *arXiv preprint arXiv:2607.24762*, 2026.
- [3] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. Mle-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024. ICLR.
- [4] Hui Chen, Miao Xiong, Yujie Lu, Wei Han, Ailin Deng, Yufei He, Jiaying Wu, Yibo Li, Yue Liu, and Bryan Hooi. Mlr-bench: Evaluating ai agents on open-ended machine learning research. *arXiv preprint arXiv:2505.19955*, 2025. NeurIPS 2025 Datasets and Benchmarks Track.
- [5] Jiefeng Chen, Bhavana Dalvi Mishra, Jaehyun Nam, Rui Meng, Tomas Pfister, and Jinsung Yoon. Mars: Modular agent with reflective search for automated ai research. *arXiv preprint arXiv:2602.02660*, 2026. ICML 2026.
- [6] Jiu hai Chen and Jonas Mueller. Automated data curation for robust language model fine-tuning. *arXiv preprint arXiv:2403.12776*, 2024.
- [7] Lichang Chen, Shiyang Li, Jun Yan, Hai Wang, Kalpa Gunaratna, Vikas Yadav, Zheng Tang, Vijay Srinivasan, Tianyi Zhou, Heng Huang, et al. AlpaGasus: Training a better alpaca with fewer data. *arXiv preprint arXiv:2307.08701*, 2023.
- [8] Wanyi Chen, Xiao Yang, Xu Yang, Tianming Sha, Qizheng Li, Zhuo Wang, Bowen Xian, Fang Kong, Weiqing Liu, and Jiang Bian. Agent² rl-bench: Can llm agents engineer agentic rl post-training? *arXiv preprint arXiv:2604.10547*, 2026.
- [9] Wentao Chen, Jiaze Zhu, Qi Fan, Yehan Ma, and An Zou. CUDA-LLM: LLMs can write efficient CUDA kernels. *arXiv preprint arXiv:2506.09092*, 2025.
- [10] Yizhe Chi, Deyao Hong, Dapeng Jiang, Tianwei Luo, Kaisen Yang, Boshi Zhang, Zhe Cao, Xiaoyan Fan, Bingxiang He, Han Hao, Weiyang Jin, Dianqiao Lei, Qingle Liu, Houde Qian, Bowen Wang, Situ Wang, Youjie Zheng, Yifan Zhou, Calvin Xiao, Eren Cai, and Qinhui Na. Frontier-eng: Benchmarking self-evolving agents on real-world engineering tasks with generative optimization. *arXiv preprint arXiv:2604.12290*, 2026. <https://arxiv.org/abs/2604.12290>.
- [11] Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, et al. UltraFeedback: Boosting language models with scaled AI feedback. *arXiv preprint arXiv:2310.01377*, 2023. ICML 2024.
- [12] Weinan Dai, Hanlin Wu, Qiying Yu, Huan-ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiaze Chen, et al. CUDA Agent: Large-scale agentic RL for high-performance CUDA kernel generation. *arXiv preprint arXiv:2602.24286*, 2026.
- [13] Stefan Falkner, Aaron Klein, and Frank Hutter. BOHB: Robust and efficient hyperparameter optimization at scale. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1437–1446, 2018.
- [14] Simin Fan, Matteo Pagliardini, and Martin Jaggi. DoGE: Domain reweighting with generalization estimation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 12895–12915, 2024.
- [15] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszczyk, et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610:47–53, 2022.
- [16] Fabio Ferreira, Lucca Wobbe, Arjun Krishnakumar, Frank Hutter, and Arber Zela. Can llms beat classical hyperparameter optimization algorithms? a study on autoresearch. *arXiv preprint arXiv:2603.24647*, 2026.
- [17] Matthias Feurer, Aaron Klein, Katharina Eggenberger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

- [18] Jonathan Gabor, Jayson Lynch, and Jonathan Rosenfeld. Evilgenie: A reward hacking benchmark. *arXiv preprint arXiv:2511.21654*, 2025.
- [19] Anson Ho, Tamay Besiroglu, Ege Erdil, David Owen, Robi Rahman, Zifan Carl Guo, David Atkinson, Neil Thompson, and Jaime Sevilla. Algorithmic progress in language models. *arXiv preprint arXiv:2403.05812*, 2024.
- [20] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- [21] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. Mlagentbench: Evaluating language agents on machine learning experimentation. In *International Conference on Machine Learning (ICML)*, 2024. <https://arxiv.org/abs/2310.03302>.
- [22] Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M. Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, et al. Population based training of neural networks. *arXiv preprint arXiv:1711.09846*, 2017.
- [23] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.
- [24] Jiajie Jin, Yuyang Hu, Kai Qiu, Qi Dai, Chong Luo, Guanting Dong, Xiaoxi Li, Tong Zhao, Xiaolong Ma, Gongrui Zhang, et al. Toward generalist autonomous research via hypothesis-tree refinement. *arXiv preprint arXiv:2606.11926*, 2026.
- [25] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [26] Andrej Karpathy. autoresearch: Ai agents running research on single-gpu nanochat training automatically. <https://github.com/karpathy/autoresearch>, 2026.
- [27] Zaid Khan, Elias Stengel-Eskin, Jaemin Cho, and Mohit Bansal. DataEnvGym: Data generation agents in teacher environments with student feedback. *arXiv preprint arXiv:2410.06215*, 2024. ICLR 2025 Spotlight.
- [28] Patrick Tser Jern Kon, Jiachen Liu, Xinyi Zhu, et al. Exp-bench: Can ai conduct ai research experiments? *arXiv preprint arXiv:2505.24785*, 2025.
- [29] Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. Towards robust agentic CUDA kernel benchmarking, verification, and optimization. *arXiv preprint arXiv:2509.14279*, 2025.
- [30] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.
- [31] Lena Libon, Ben Rank, Jehyeok Yeon, et al. Researcharena: Evaluating sabotage and monitoring in automated ai r&d. *arXiv preprint arXiv:2607.19321*, 2026.
- [32] Wei Liu, Weihao Zeng, Keqing He, Yong Jiang, and Junxian He. What makes good data for alignment? a comprehensive study of automatic data selection in instruction tuning. *arXiv preprint arXiv:2312.15685*, 2023. ICLR 2024.
- [33] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- [34] Xinyu Lu, Tianshu Wang, Pengbo Wang, et al. The meta-agent challenge: Are current agents capable of autonomous agent development? *arXiv preprint arXiv:2606.04455*, 2026.
- [35] Yujie Luo, Xiangyuan Ru, Jingsheng Zheng, Jingjing Wang, Yuqi Zhu, Jintian Zhang, Runnan Fang, Kewei Xu, Ye Liu, Zheng Wei, et al. Exploring autonomous agentic data engineering for model specialization. *arXiv preprint arXiv:2605.30407*, 2026.
- [36] Bohan Lyu, Yucheng Yang, Siqiao Huang, et al. Mls-bench: A holistic and rigorous assessment of ai systems on building better ai. *arXiv preprint arXiv:2605.08678*, 2026.
- [37] Yanuo Ma, Ben Kereopa-Yorke, and Ben Schultz. Building to the test: Coding agents deliver what you check, not what you requested. *arXiv preprint arXiv:2606.28430*, 2026.

- [38] Daniel J. Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, Shariq Iqbal, Jean-Baptiste Lespiau, Alex Ahern, et al. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618:257–263, 2023.
- [39] Fanqing Meng, Lingxiao Du, Qiguang Chen, Ziqi Zhao, Haocheng Lu, Mengkang Hu, and Michael Qizhe Shieh. Rsibench-data: Benchmarking data-centric research for recursive self-improvement. *arXiv preprint arXiv:2607.25886*, 2026.
- [40] Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, Dieuwke Hupkes, Ricardo Silveira Cabral, Tatiana Shavrina, Jakob Foerster, Yoram Bachrach, William Yang Wang, and Roberta Raileanu. Mlgym: A new framework and benchmark for advancing ai research agents. *arXiv preprint arXiv:2502.14499*, 2025. <https://arxiv.org/abs/2502.14499>.
- [41] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 47356–47415, 2025.
- [42] Rushi Qiang, Yuchen Zhuang, Yinghao Li, et al. Mle-dojo: Interactive environments for empowering llm agents in machine learning engineering. *arXiv preprint arXiv:2505.07782*, 2025.
- [43] Ben Rank, Hardik Bhatnagar, Ameya Prabhu, Shira Eisenberg, Karina Nguyen, Matthias Bethge, and Maksym Andriushchenko. Posttrainbench: Can llm agents automate llm post-training? *arXiv preprint arXiv:2603.08640*, 2026.
- [44] Bernardino Romera-Paredes, Mohammadamin Barekatin, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- [45] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. The curse of recursion: Training on generated data makes models forget. *arXiv preprint arXiv:2305.17493*, 2023.
- [46] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, et al. Paperbench: Evaluating ai’s ability to replicate ai research. *arXiv preprint arXiv:2504.01848*, 2025.
- [47] Xiangru Tang, Yuliang Liu, Zefan Cai, et al. Ml-bench: Evaluating large language models and agents for machine learning tasks on repository-level code. *arXiv preprint arXiv:2311.09835*, 2023.
- [48] Edan Toledo, Karen Hambardzumyan, Martin Josifoski, Rishi Hazra, Nicolas Baldwin, Alexis Audran-Reiss, Michael Kuchnik, Despoina Magka, Minqi Jiang, Alisia Maria Lupidi, et al. AI research agents for machine learning: Search, exploration, and generalization in MLE-bench. *arXiv preprint arXiv:2507.02554*, 2025.
- [49] Pablo Villalobos, Anson Ho, Jaime Sevilla, Tamay Besiroglu, Lennart Heim, and Marius Hobbhahn. Will we run out of data? limits of llm scaling based on human-generated data. *arXiv preprint arXiv:2211.04325*, 2022.
- [50] Hao Wang, Hanchen Li, Qiuyang Mang, Alvin Cheung, Koushik Sen, and Dawn Song. Do androids dream of breaking the game? systematically auditing ai agent benchmarks with benchjack. *arXiv preprint arXiv:2605.12673*, 2026.
- [51] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-Instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*, 2022. ACL 2023.
- [52] Hjalmar Wijk, Tao Lin, Joel Becker, et al. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*, 2024.
- [53] Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. LESS: Selecting influential data for targeted instruction tuning. *arXiv preprint arXiv:2402.04333*, 2024. ICML 2024.
- [54] Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V. Le, Tengyu Ma, and Adams Wei Yu. DoReMi: Optimizing data mixtures speeds up language model pretraining. *arXiv preprint arXiv:2305.10429*, 2023. NeurIPS 2023.
- [55] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. WizardLM: Empowering large pre-trained language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023. ICLR 2024.

- [56] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.
- [57] Junlin Yang, Che Jiang, Yu Fu, et al. Frontis-ma1: Training an ai4ai model towards recursive self-improvement in machine learning engineering. *arXiv preprint arXiv:2607.28568*, 2026.
- [58] Lingyun Yang, Yuxiao Wang, Shenghao Liang, Linfeng Yang, Daocheng Ying, Chunbo You, Rui Zhang, Luping Wang, Yinghao Yu, Guodong Yang, et al. Are LLM-generated GPU kernels production-ready? a trace-driven benchmark and optimization agent. *arXiv preprint arXiv:2607.14541*, 2026.
- [59] Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Hu. Data-centric artificial intelligence: A survey. *arXiv preprint arXiv:2303.10158*, 2023.
- [60] Yunxiang Zhang, Muhammad Khalifa, Shitanshu Bhushan, Grant D Murphy, Lajanugen Logeswaran, Jaekyeom Kim, Moontae Lee, Honglak Lee, and Lu Wang. Mlrc-bench: Can language agents solve machine learning research challenges? *arXiv preprint arXiv:2504.09702*, 2025. NeurIPS 2025 Datasets and Benchmarks Track.
- [61] Zijian Zhang, Rong Wang, Shiyang Li, Yuebo Luo, Mingyi Hong, and Caiwen Ding. CudaForge: An agent framework with hardware feedback for CUDA kernel optimization. *arXiv preprint arXiv:2511.01884*, 2025.

A The ten tasks

Each task freezes one research repository and asks for the same thing: improve the training algorithm that repository applies to its own model. What differs between them is the algorithm, the asset the agent is given, and the pair of metrics either side of the evaluation boundary — a cheap one it may query as often as it likes during its four hours, and the one that decides its result, computed afterwards by an evaluator it never sees. The two are related differently on different tasks, and the relation matters when reading a column: on some the proxy is a subsample of the final protocol, on others it is a different benchmark entirely.

OpenR1 — supervised fine-tuning. Qwen2.5-Coder-1.5B-Instruct is fine-tuned on a decontaminated 8,005-row Python CodeForces projection; the shipped recipe is completion-only supervised fine-tuning with the prompt tokens masked. A candidate may select, reweight, pack, transform or synthesise training signal from those rows, and may change the masking or the objective. The proxy is `livecodebench_public_pass_at_1`; the final metric is the whole LiveCodeBench v6 release slice, 175 problems under the benchmark’s own sampling protocol — ten samples per problem at temperature 0.2 and top- p 0.95, capped at 2,048 new tokens, scored as the mean over problems of the fraction of samples that pass every official test.

RAGEN — multi-turn agentic RL. A Qwen2.5-3B-Instruct policy is trained on Sokoban with multi-turn on-policy GRPO, generating its own boards and trajectories online. Board construction, curriculum, rollout collection, reward shaping and the update rule are all open; the scoring engine, the action decoding and the final seeds are not. The proxy is a four-bank solve rate, the final metric a held-out 512-board solve rate, and the two use different fixed environment-seed protocols.

OPD — on-policy distillation. A 1.5B student is distilled from a mounted teacher by sampled-token on-policy distillation. The proxy and the final metric are different benchmarks rather than two views of one: the proxy is MATH-500 at four samples per question with a 12,288-token cap, and the final metric is AIME 2024 and 2025, 60 questions at 32 samples with a 31,744-token cap. MATH-500 is mounted during exploration; the AIME inputs are not.

BTRM — Bradley–Terry reward modelling. A scalar reward model is trained from a fixed Mistral-7B start on decontaminated UltraFeedback preference pairs under a Bradley–Terry objective. The artifact must remain loadable as a scalar reward model on that architecture, and any overlap between a training row and RewardBench invalidates the run. Here the proxy is a strict subsample of the final: 512 of the 2,985 pairs are visible, and the remaining 2,473 are held out until scoring.

DPO — preference optimization. A merged Zephyr/Mistral-7B model is aligned with direct preference optimization. The final metric is IFEval prompt-level strict accuracy over 413 held-out prompts.

DDPO — diffusion RL. Stable Diffusion v1.5 is fine-tuned with on-policy DDPO and a LoRA adapter against a frozen CLIP aesthetic reward. Prompt construction and sampling, reward shaping and normalization, auxiliary losses, the update rule and the trainable parameters are all open; the reward assets and the final prompt/latent stream are fixed, the latter mounted only at scoring. The proxy scores 64 generated images, the final metric 256. CLIP alignment and mean pairwise image distance are reported alongside it.

NPO — machine unlearning. Llama-3.2-1B-Instruct is unlearned on the TOFU **forget10** protocol with the official OpenUnlearning NPO recipe. Final evaluation reports two pinned components, an extraction strength that is better lower and a model utility that is better higher; the scalar we compare on is their harmonic mean after normalising each against the training start and a retain-90 reference. During exploration only the published anchor and the train-role projection are mounted — the retain-90 anchor and the final-role data appear only in the separate score phase.

DiGress — discrete graph diffusion. A discrete graph diffusion model is trained on QM9 without hydrogens. The fast metric is the product of validity, uniqueness and novelty rates, higher better; the final metric is the

upstream test negative log-likelihood, lower better, and the repository’s validation NLL is what connects them. The real test split is mounted only at scoring.

Model Soup — weight averaging. Seventy-two fixed CLIP ViT-B/32 checkpoints are combined into one model; the shipped construction is a uniform mean. What the submitted code decides is which ingredients to use and with what coefficients, and those coefficients may be negative or extrapolative. The proxy is 2,000 ImageNet-V2 images and the final metric the full 10,000.

OWL — one-shot pruning. Unstructured sparsity is imposed on a dense OPT-6.7B in a single pass by activation-aware OWL/Wanda pruning, with no fine-tuning in the shipped recipe. The hard artifact gate is decoder sparsity within $[0.699, 0.701]$; within that window the pruning criterion, the search, the use of the mounted C4 calibration shard, and any training a candidate cares to add are open. The proxy is WikiText-2 validation perplexity and the final metric WikiText-2 test perplexity, the test text mounted only at scoring.

Two of the ten ship a procedure that trains nothing: weight averaging combines checkpoints handed over as data, and one-shot pruning removes weights from a released model in a single pass. Nothing in the contract requires a submission to leave it that way, and on both tasks some did not — the submissions read in §4.3 add a distillation fine-tune to the pruning pipeline and a gradient-based search to the soup, and use the twelve hours accordingly. The two are kept because the algorithmic question is as real in them as anywhere else: which checkpoints to combine and how, which weights to remove and by what criterion.